

CS6.302 – Software System Development

Assignment 2 – : Front End Design

Total Marks: 100

Duration: 3 Weeks

Release Date: 16 September 2026

Due Date: 15 October 2026

About the Assignment

- This is a **Team Assignment** with a team size of **4**.
- **Teams have been re-formed for Assignment 2.** Your new team number and members are published on Moodle.
- Each new team **inherits a Assignment 1 submission** (the PostgreSQL + MongoDB backend for one of the five projects) and builds a front end on top of it.
- Submissions are accepted via Moodle and your team's shared GitHub repository.
- All questions are self-explanatory. All assumptions are open to your interpretation. **List your assumptions** in the README.

Why re-form teams?

In real projects you often continue work that someone else started. Assignment 2 simulates this: you will read and understand a database designed by another team, fix what needs fixing, and design a user interface that lets people actually use it.

Inheriting a Assignment 1 Submission

1. Your Assignment 2 project number is given in teams excel file.
2. Course staff will assign each team one Assignment 1 repository for that project. The mapping is published on Moodle.
3. Fork the assigned repository into your new team's GitHub account and continue development there. Keep the Assignment 1 commit history intact.

4. You may fix or improve the inherited database scripts. Note every change you make in your README.
 5. If the inherited repository does not run at all or does not open, inform the TAs within 48 hours.
-

Objective

Design and build a **web front end** for your inherited project so that a user can perform the four Part A workflows through a browser instead of running scripts by hand. You will need a small server/API layer so the browser can talk to PostgreSQL and MongoDB; keep it simple and let the database (stored procedures, triggers, views, aggregation pipelines) do the real work.

You are free to choose **any** front-end framework, styling approach, chart/map libraries, and server technology your team is comfortable with. Justify your choices briefly in the README.

Step 1: Understand What You Inherited

- Get the Part A schema, seed data, procedures, and pipelines running locally using the original team's README.
 - Write a short **handover note** ([docs/handover_note.md](#) , about one page) covering: what worked, what did not, what you changed and why, and any suggestions you would give the original authors.
 - Sketch the API you need (a simple list of endpoints with inputs and outputs is enough) before you start building the UI.
-

Step 2: Design Before You Build

- Identify the **main users** of your application (e.g., for BiteStream: a customer and a restaurant manager) and describe what each of them wants to do.
 - Draw **wireframes** for your screens (hand-drawn, Figma, Excalidraw - anything is fine). Place them in [docs/design/](#) .
 - Decide on a consistent look: colours, fonts, and a small set of reusable components (buttons, forms, tables, cards). Consistency matters more than sophistication.
-

Step 3: Build the Screens

Your application must expose all four Assignment 1 workflows plus basic browsing. The suggested screens are below; you may organise them differently as long as the functionality is covered.

1. Browse

List the main entities of your project (e.g., users, restaurants, orders) with pagination, basic search/filter, and a detail view per record. Do not load all 50,000+ rows into the browser at once.

2. Transaction (Workflow 1)

A form that calls the Assignment 1 stored procedure (checkout / booking / appointment / gig funding).

- Show the actor's balance before and after.
- When the procedure fails (insufficient balance, or a second active order/trip/stay/consultation/gig blocked by the partial unique index), show a friendly message explaining what went wrong.
- After success, show the new `wallet_audit_logs` entry so it is clear the trigger fired.

3. Audit Trail

A read-only view of `wallet_audit_logs` for a chosen user/rider/guest/patient/client, with a date filter and a running balance.

4. Analytics (Workflow 2 + Materialized View)

- A chart of the **7-day moving average** revenue per restaurant/vehicle/property/clinic/freelancer, using the Part A window-function query.
- A ranked table based on `DENSE_RANK()`.
- Summary numbers from the **materialized view**, with a "last refreshed" time and a button to refresh it.

5. Map (Workflow 3)

A map showing the results of the `$geoNear` pipeline around a selectable origin, with a 5 km radius and a list of results sorted by distance. New pings should appear without a full page reload (polling is fine).

6. Reviews (Workflow 4)

Charts driven by the `$facet` pipeline: rating distribution, top tags, and overall average rating. Also display at least one flexible catalog document (Menus / VehicleMetadata / PropertyAmenities /

MedicalCatalogs / Portfolios) in a readable way.

Project mapping

Screen	P1 BiteStream	P2 RideSync	P3 StaySpot	P4 CareConnect	P5 GigTask
Browse	users, restaurants, orders	riders, vehicles, trips	guests, properties, bookings	patients, clinics, appointments	clients, freelancers, contracts
Transaction	place order	book trip	book stay	book appointment	fund gig
Audit	wallet	wallet	wallet	HSA	escrow
Analytics	per restaurant	per vehicle	per property	per clinic	per freelancer
Map	nearest driver to restaurant	nearest vehicle to rider	search hotspots	nearest nurse to patient	nearest worker to job site
Reviews + catalog	Reviews + Menus	TripReviews + VehicleMetadata	PropertyReviews + PropertyAmenities	PatientReviews + MedicalCatalogs	GigReviews + Portfolios

General expectations

- Every screen should handle **loading**, **empty**, and **error** situations gracefully - no blank pages.
 - The application should be usable on a laptop screen and reasonably usable on a phone.
 - Provide a simple way to switch between users/actors for demo purposes (real login is not required).
 - Provide a way to move an order/trip/booking/appointment/contract through its statuses so the partial unique index can be demonstrated.
-

Step 4: Wrap Up

- Write **a few tests** for the parts you consider most important (for example, the transaction form's success and failure cases). We care that you tested thoughtfully, not about coverage numbers.
 - Make sure a new person can run the whole system from your README. A `docker-compose.yml` is recommended but not mandatory.
 - Record a **short demo video (≤ 5 minutes)** walking through the screens against the seeded data; link it in the README.
-

Submission Guidelines

Suggested repository structure (Assignment 1 folders are retained; Assignment 2 folders are added):

```
<new_team_number>_alb/
├── README.md                # setup steps, assumptions, tech choices,
                              # screenshots,
                              # contribution table, repo URL + final
                              # commit hash, demo link
├── docs/
│   ├── handover_note.md
│   ├── api_endpoints.md
│   └── design/              # wireframes, user descriptions
├── sql/  mongo/  data_generation/  # from Part A (+ your fixes)
├── api/
└── web/                     # your server layer
                             # your front end
```

In Moodle

- 1. Submit a single ZIP named `<new_team_number>_alb.zip` (strictly under **20 MB**). If any team has multiple submissions/ Has wrong naming scheme they will be penalised.
- 2. **DO NOT** include `node_modules/` , `venv/` , `__pycache__/` , build output, or database dumps.
- 3. Ensure `README.md` contains the GitHub repository URL and the final commit hash.

Grading Rubric (100 Points Total)

Component	Points	What we look for
Handover & Understanding	10	Inherited system runs; handover note is honest and specific; fixes are documented.
Design	15	Clear idea of who the users are; wireframes; consistent, clean look across screens.
Workflow Screens	30	All four workflows work through the UI and are correctly wired to the Part A procedure, query, and pipelines; failures are shown meaningfully.
Front-End Quality	15	Browsing at scale, loading/empty/error handling, reasonable responsiveness, sensible code organisation.
Documentation & Demo	5	README, assumptions, tech justification, demo video.

Component	Points	What we look for
Viva Voce	25	10 points technical questioning (any member, any part - including the inherited database code); 15 points live demonstration.

Notes

- Every team member is expected to contribute to both design and code. Include a contribution table in the README; marks may be scaled for very unequal contribution.
- Late submission: -10% per day, up to 3 days.

Questions: post on the Moodle forum under "Assignment 2" so that answers are visible to all teams.